3 Chapter 3: Time-Domain Analysis of Discrete-Time Systems

3.1 Introduction

Discrete-time signals, unlike continous-time signals, have a discretized domain n rather than a continous domain t, and have a range specified by $x[n]$. Any sequence of numbers can be considered a discrete-time signal. Though discrete-time signals and their corresponding discrete-time systems have similar general properties to the continuous-time systems we analyzed in Chapter 2, there are a number of important difference that we'll learn about in this chapter.

It's worth noting that discrete-time signals often arise as a result of *sampling* continous-time signals.

Uniform sampling is ideal, and the only process we will be working with in this class, meaning that the continuous-time signal will be sampled at a consistent period of T, such that $x[n] = x(nT)$.

Sampling and *quantization* are different processes. Sampling occurs on the t-axis and picks out period values of a continuous signal, converting it to a discrete-time signal. Quantization occurs on the y-axis, and converts an analog signal to a digital one by reducing its real value to the closest available digital (pick from a finite array of options) value.

For discrete-time signals, the measurement of energy and power are almost identical to that for continuous time signals:

$$E_x = \sum_{n=\infty}^{\infty} |x[n]|^2$$

$$P_x = \lim_{N\to\infty} \frac{1}{2N+1} \sum_{-N}^{N} |x[n]|^2.$$

Similar to continuous-time signals, periodic discrete-time signals may have their power calculated over a single period N_0 rather than taking the limit as $N \to \infty$. For discrete-time signals, we may define a *fundamental period*, the smallest value of N_0 that satisfies

$$x[n] = x[n + N_0].$$

3.2 Useful Signal Operations

Standard signal operations (time-shifting, time-reversal) are similar to those in continuous-time signals:

$$x_s[n] = x[n-5]$$
$$x_r[n] = x[-n].$$

The discrete analogue of compression and expansion of continuous-time signals is downsampling and upsampling. Since discrete-time signals rely on a finite number of values, compression and expansion result in *less* or *more* values. In downsampling (compression), every other sample is deleted, and the signal is whittled down to $\frac{n}{2}$ samples. In upsampling (expansion), intermediate values are inserted for a total of $2n$ samples. We could set the intermediate, added samples to 0, but it makes more sense to artificially interpolate the average value of the two adjacent samples at each added sample. It's important to remember that we haven't added any real data here that represents a real-world value, but we haven't lost any either (like we do in downsampling).

3.3 Some Useful Discrete-Time Signal Models

Similar to continous time signals, there are a number of useful signal models that correspond to their continuous-time counterparts.

The discrete-time version of the impulse function $\delta[n]$ is defined as

$$\delta[n] = \begin{cases} 1 & n = 0 \\ 0 & n \neq 0 \end{cases}$$

Note that there is no infinitely-valued pulse for this version of the impulse function.

The unit step, similarly, is defined as

$$u[n] = \begin{cases} 1 & \text{for } n \geq 0 \\ 0 & \text{for } n < 0. \end{cases}$$

The discrete-time exponential, analogous to the continuous-time exponential, $e^{\lambda t}$, requires some explanation. Though the fundamental idea is the same, rather than converting directly to the discrete time domain as $e^{\lambda n}$, we will use γ^n, where $\gamma = e^{\lambda}$.

Discrete-time sinusoids are straightforward, and may be expressed as

$$C\cos(\Omega n + \theta).$$

Extending to complex exponentials, we may then express a complex exponential using Euler's formula as

$$e^{j\Omega n} = \cos(\Omega n) + j\sin(\Omega n).$$

3.4 Examples of Discrete-Time Systems

As examples of discrete-time systems were covered in depth in Chapter 1, we won't revisit them here. However, it is worth pointing out a concept analogous to the differential equation in continuous-time systems, the **difference equation**. A comparison between the two is better served by an example.

Given a simple, first-order differential equation

$$\frac{dy(t)}{dt} + cy(t) = x(t),$$

we will consider uniformly sampling $x(t)$ at intervals of T seconds, transforming it into $x[n]$. We'll do the same for $y(t) = y[n]$. Then, using the definition of the derivative with a discrete-time twist,

$$\lim_{T \to 0} \frac{y[n] - y[n-1]}{T} + cy[n] = x[n].$$

Simplifying,

$$y[n] + \alpha y[n-1] = \beta x[n],$$

where

$$\alpha = \frac{-1}{1+cT} \qquad \text{and} \qquad \beta = \frac{T}{1+cT}.$$

It will be convenient later on to use what we will refer to as the "advance" form of a difference equation, which can be accomplished by simply time shifting both sides of the equation such that $y[n]$ is the lowest order sample in the equation. For example, delaying the above example by 1,

$$y[n+1] + \alpha y[n] = \beta x[n+1].$$

As we'll see, this notation makes it easier to make certain assumptions about the memory requirements of the discrete-time system under consideration.

Similar to continuous-time signals, discrete-time signals can be categorized as linear/non-linear, time-variant/time-invariant, and so on.

3.5 Discrete-Time System Equations

Now that the groundwork is laid for discrete-time signals and their corresponding systems, we can move on to develop a procedure for finding a discrete-time system's ZIR and ZSR, where its total response will also be the sum of the two. As noted before, we will be working with difference equations rather than differential equations in this chapter, but we will still be sticking to *linear* difference equations with constant coefficients.

We have the choice of using the advance and delay forms of a difference equation, but we'll be working primarily with the advance form for convenience (the choice is somewhat arbitrary). For reference, the standard form of an N, M-order difference equation is

$$y[n+N] + a_1 y[n+N-1] + \ \ldots \ + a_{N-1} y[n+1] + a_N y[n] =$$
$$b_0 x[n+N] + b_1 x[n+N-1] + \ \ldots \ + b_{N-1} x[n+1] + b_N x[n].$$

As with the continous-time discussion, $M = N$ at most, and more often than not, $M < N$; both ensure that the system remains causal. Generally, we will normalize the coefficient of the highest order term, a_0, to 1, by dividing it and all the other coefficients by the appropriate value. This ensures that we can safely time-shift terms, as in the initial example, without causing problems.

To solve a difference equation, we can resort to an iterative or recursive method. Similar to a continuously-valued differential equation, the solution to $y[0]$ requires knowledge of some initial conditions; that is, the system's prior outputs (if we use the delay form of the difference equation) $y[n-1], y[n-2], \ \ldots \ y[-N]$, and the right hand side can be solved similarly. Therefore, a recursive or iterative method is often appropriate to solve a given difference equation, where previous values of the input/output are used to determine the corresponding value at $n = 0$. Then, each future step can also be solved for using the previous step, and so on, as long as we have N initial conditions available to us to solve for y[0].